Experiment No. 7

Motion Estimation using Optical Flow Algorithms
in Video Sequences

Aim

To implement optical flow algorithms for estimating the motion of objects between consecutive video frames and analyze motion patterns for dynamic scene understanding using Python and OpenCV.

Purpose

Optical flow is a fundamental computer vision technique used to estimate the apparent motion of objects, surfaces, or camera movement by analyzing consecutive frames in a video sequence. Motion estimation plays a vital role in autonomous driving, video surveillance, robotics, gesture recognition, sports analytics, and augmented reality.

In this experiment, Lucas-Kanade Sparse Optical Flow and Farneback Dense Optical Flow are implemented to estimate object motion, visualize motion vectors, and compare the two techniques.

Prerequisites

- Basic knowledge of Digital Image Processing
- Familiarity with Python programming
- Python libraries: OpenCV (cv2), NumPy, and Matplotlib
- Understanding of video processing and image gradients
- Jupyter Notebook, Google Colab, or Visual Studio Code

Steps (Procedure)

1. Import the required libraries and load a video sequence or access a webcam stream.
2. Read consecutive frames and convert them to grayscale.
3. Implement Lucas-Kanade optical flow to estimate the motion of selected feature points.
4. Visualize the motion vectors or trajectories of the tracked points.
5. Implement Farneback dense optical flow to estimate motion for every pixel.
6. Display the dense flow using color-coded visualization of direction and magnitude.
7. Compare Lucas-Kanade and Farneback in terms of accuracy, computational complexity, and robustness.
8. Analyze the effect of object speed, lighting conditions, and camera motion.
9. Evaluate the suitability of each algorithm for tracking, surveillance, and navigation.
10. Summarize the observations and discuss the significance of optical flow.

Key Concepts and Formulas

- **Brightness constancy assumption:** $I(x, y, t) = I(x + dx, y + dy, t + dt)$.
- **Optical flow constraint equation** (first-order Taylor expansion): $I_x\, u + I_y\, v + I_t = 0$, where (u, v) is the flow and I_x, I_y, I_t are the spatial and temporal derivatives. One equation, two unknowns: the **aperture problem**.

- **Lucas-Kanade:** assumes (u, v) is constant in a small window; stacking the equations for all window pixels gives A d = b, solved by least squares, d = $(A^T A)^{-1} A^T$ b. The matrix $A^T A$ needs to be well conditioned (i.e., the window contains corners or texture), which is why Shi-Tomasi corners are tracked.
- **Pyramids:** the image is processed from coarse to fine scale so that large motions become small at the coarse levels.
- **Farneback:** approximates each pixel neighborhood with a quadratic polynomial and estimates the displacement from how the polynomial coefficients change between frames, giving a flow vector for every pixel.
- **Colour coding:** hue = direction of motion, brightness = speed. Quality measure on synthetic data: average end-point error (EPE) = mean of ||estimated flow - true flow||.

Observations

- Lucas-Kanade tracks only selected corner points, so it is very fast (a few milliseconds per frame pair). Most points lie on the static background and have near-zero displacement; points on the moving object show clear trajectories in the object's direction.
- Farneback gives a flow vector for every pixel, so the moving object appears as a uniformly colored region against a dark (static) background. It is slower per frame pair than Lucas-Kanade, in this run roughly 10 times slower.
- On the synthetic test video (object moving 4 px right and 2 px down per frame), Farneback recovered the motion almost exactly (end-point error under 0.1 px inside the object), with almost no false motion on the static background.
- Speed matters. Without an image pyramid the estimate failed once the motion exceeded the search window (around 12 px per frame in this test, error about equal to the true displacement), while with 3 pyramid levels it stayed accurate up to at least 40 px per frame.
- The brightness constancy assumption is violated by lighting changes, shadows, and reflections, and flow quality is poor in untextured regions (aperture problem). The synthetic video is strongly textured, so it showed little effect here; test with your real video to observe these effects.
- Camera motion makes the whole background move, so moving objects must be found as flow that differs from the dominant (global) motion.

Source Code

File: experiment7_optical_flow.py

```
"""
Experiment 7: Motion Estimation using Optical Flow (Lucas-Kanade and Farneback)
Install:  pip install opencv-python numpy matplotlib
Usage:    python experiment7_optical_flow.py [video_path | 0]
          video_path : a video file          0 : live webcam
          (no argument: a synthetic video with known motion is generated)
"""
import os
import sys
import time
import cv2
import numpy as np
import matplotlib.pyplot as plt

os.makedirs("output", exist_ok=True)
rng = np.random.default_rng(5)
N_FRAMES = 40

# ---------------------------------------------------------------
# Step 1: Video source
# ---------------------------------------------------------------
src = sys.argv[1] if len(sys.argv) > 1 else None
true_motion = None  # (dx, dy) per frame, known only for the synthetic video


def synthetic_frames(n=N_FRAMES, dx=4, dy=2):
    """Textured static background + a textured square moving by (dx, dy) px per frame."""
    h, w = 300, 400
    bg = cv2.GaussianBlur(rng.integers(0, 255, (h, w), dtype=np.uint8), (5, 5), 0)
    obj = cv2.GaussianBlur(rng.integers(60, 255, (70, 70), dtype=np.uint8), (3, 3), 0)
    for i in range(n):
        f = bg.copy()
        x, y = 40 + dx * i, 60 + dy * i
        f[y:y + 70, x:x + 70] = obj
        f = np.clip(f + rng.normal(0, 4, f.shape), 0, 255).astype(np.uint8)   # sensor noise
        yield cv2.cvtColor(f, cv2.COLOR_GRAY2BGR)


if src is None:
    print("No video given - generating a synthetic video (object moves 4 px right, 2 px down per frame).")
    frames = list(synthetic_frames())
    true_motion = (4, 2)
else:
    cap = cv2.VideoCapture(0 if src == "0" else src)
    frames = []
    while len(frames) < 300:
        ok, fr = cap.read()
        if not ok:
            break
        frames.append(cv2.resize(fr, (640, int(fr.shape[0] * 640 / fr.shape[1]))))
    cap.release()
    if len(frames) < 2:
        sys.exit("Could not read at least two frames from the video source.")

print(f"Frames loaded: {len(frames)}  size: {frames[0].shape[1]}x{frames[0].shape[0]}")

# ---------------------------------------------------------------
# Step 2: Grayscale (optical flow uses intensity, not colour)
# ---------------------------------------------------------------
grays = [cv2.cvtColor(f, cv2.COLOR_BGR2GRAY) for f in frames]

# ---------------------------------------------------------------
# Steps 3 & 4: Lucas-Kanade sparse optical flow with trajectories
# ---------------------------------------------------------------
feature_params = dict(maxCorners=100, qualityLevel=0.2, minDistance=7, blockSize=7)
lk_params = dict(winSize=(21, 21), maxLevel=3,
                 criteria=(cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 30, 0.01))

p0 = cv2.goodFeaturesToTrack(grays[0], mask=None, **feature_params)   # Shi-Tomasi corners
tracks = [[tuple(pt.ravel())] for pt in p0]                           # trajectory per point
trail_canvas = np.zeros_like(frames[0])
colors = rng.integers(0, 255, (200, 3)).tolist()

t_lk = 0.0
prev_pts = p0
lk_vis = frames[0].copy()
for i in range(1, len(grays)):
    t0 = time.perf_counter()
    p1, st, err = cv2.calcOpticalFlowPyrLK(grays[i - 1], grays[i], prev_pts, None, **lk_params)
```

```
    # forward-backward consistency check removes unreliable tracks
    p0r, st2, _ = cv2.calcOpticalFlowPyrLK(grays[i], grays[i - 1], p1, None, **lk_params)
    t_lk += time.perf_counter() - t0
    fb_err = np.linalg.norm(prev_pts - p0r, axis=2).ravel()
    good = (st.ravel() == 1) & (st2.ravel() == 1) & (fb_err < 1.0)

    new_tracks, new_pts = [], []
    for k in np.where(good)[0]:
        a = tuple(map(int, prev_pts[k].ravel())); b = tuple(map(int, p1[k].ravel()))
        cv2.line(trail_canvas, a, b, colors[k % 200], 2)
        tracks[k].append(tuple(p1[k].ravel()))
    prev_pts = p1[good].reshape(-1, 1, 2)
    tracks = [tracks[k] for k in np.where(good)[0]]
    if len(prev_pts) == 0:
        break
    lk_vis = cv2.add(cv2.circle(frames[i].copy(), tuple(map(int, prev_pts[0].ravel())), 0, (0, 0, 0), 1), trail_canvas)

for pt in prev_pts:  # final point positions
    cv2.circle(lk_vis, tuple(map(int, pt.ravel())), 4, (0, 255, 255), -1)

moving = [np.array(t[-1]) - np.array(t[0]) for t in tracks if len(t) > 5]
print(f"\nLucas-Kanade: {len(p0)} points initialised, {len(prev_pts)} survived to the end")
print(f"LK mean time per frame pair: {1000 * t_lk / (len(grays) - 1):.2f} ms")
if moving:
    disp = np.array(moving)
    mag = np.linalg.norm(disp, axis=1)
    print(f"Tracks that moved >10 px in total: {(mag > 10).sum()} (the rest are static background points)")

# ---------------------------------------------------------------
# Steps 5 & 6: Farneback dense optical flow + HSV colour coding
# ---------------------------------------------------------------
def flow_to_color(flow):
    """Hue = direction, brightness = magnitude."""
    mag, ang = cv2.cartToPolar(flow[..., 0], flow[..., 1])
    hsv = np.zeros((*flow.shape[:2], 3), np.uint8)
    hsv[..., 0] = ang * 180 / np.pi / 2
    hsv[..., 1] = 255
    hsv[..., 2] = cv2.normalize(mag, None, 0, 255, cv2.NORM_MINMAX)
    return cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR), mag

fb_params = dict(pyr_scale=0.5, levels=3, winsize=15, iterations=3, poly_n=5, poly_sigma=1.2, flags=0)

mid = len(grays) // 2                      # analyse a frame pair in the middle
t0 = time.perf_counter()
flow = cv2.calcOpticalFlowFarneback(grays[mid - 1], grays[mid], None, **fb_params)
t_fb = (time.perf_counter() - t0) * 1000
flow_color, flow_mag = flow_to_color(flow)
print(f"\nFarneback: flow field shape {flow.shape} (dx, dy for every pixel)")
print(f"Farneback time per frame pair: {t_fb:.2f} ms")
print(f"Max motion magnitude: {flow_mag.max():.2f} px/frame")

# Arrow (quiver) view of the dense field on a coarse grid
step = 16
yy, xx = np.mgrid[step // 2:grays[0].shape[0]:step, step // 2:grays[0].shape[1]:step]
fx, fy = flow[yy, xx, 0], flow[yy, xx, 1]
arrows = frames[mid].copy()
for (x, y, u, v) in zip(xx.ravel(), yy.ravel(), fx.ravel(), fy.ravel()):
    if np.hypot(u, v) > 1.0:
        cv2.arrowedLine(arrows, (x, y), (int(x + 3 * u), int(y + 3 * v)), (0, 255, 0), 1, tipLength=0.4)

# ---------------------------------------------------------------
# Step 7 & 8: Quantitative evaluation (synthetic ground truth) and speed effect
# ---------------------------------------------------------------
if true_motion is not None:
    gt = np.array(true_motion, np.float32)
    h, w = grays[0].shape
    i = mid
    x0, y0 = 40 + true_motion[0] * (i - 1), 60 + true_motion[1] * (i - 1)
    region = (slice(y0 + 10, y0 + 60), slice(x0 + 10, x0 + 60))     # inside the moving object
    est = flow[region].reshape(-1, 2).mean(axis=0)
    epe = np.linalg.norm(flow[region] - gt, axis=2).mean()
    print(f"\nGround truth motion: {true_motion}   Farneback estimate inside object: ({est[0]:.2f}, {est[1]:.2f})")
    print(f"Average end-point error inside object: {epe:.3f} px")
    bg_motion = np.linalg.norm(flow[250:290, 250:390], axis=2).mean()
    print(f"Mean flow magnitude on static background: {bg_motion:.3f} px")

    # Effect of object speed, with and without an image pyramid (levels=1 means no pyramid)
    print("\n---- Effect of object speed: Farneback mean error inside object (px) ----")
    print(f"{'speed (px/frame)':<18}{'levels=1 (no pyramid)':>24}{'levels=3 (pyramid)':>22}")
    for speed in (2, 6, 12, 24, 40):
        fr = list(synthetic_frames(3, dx=speed, dy=0))
        g = [cv2.cvtColor(f, cv2.COLOR_BGR2GRAY) for f in fr]
        reg = (slice(60 + 15, 60 + 55), slice(40 + speed + 15, 40 + speed + 55))
        errs = []
        for lv in (1, 3):
```

```
            prm = dict(fb_params, levels=lv)
            fl = cv2.calcOpticalFlowFarneback(g[1], g[2], None, **prm)
            errs.append(np.linalg.norm(fl[reg] - np.array([speed, 0], np.float32), axis=2).mean())
        print(f"{speed:<18}{errs[0]:>24.2f}{errs[1]:>22.2f}")

    # Effect of lighting change: a uniform brightness offset keeps gradients intact, but a
    # spatially varying illumination change violates brightness constancy and creates false motion
    print("\n---- Effect of lighting change (static background region) ----")
    uniform = cv2.convertScaleAbs(grays[mid], alpha=1.0, beta=40)
    ramp = np.tile(np.linspace(0.5, 1.3, grays[mid].shape[1]), (grays[mid].shape[0], 1))
    gradient_light = np.clip(grays[mid] * ramp, 0, 255).astype(np.uint8)
    for name, g2 in [("no change", grays[mid]), ("uniform +40 brightness", uniform),
                     ("illumination gradient (x0.5 to x1.3)", gradient_light)]:
        fl_b = cv2.calcOpticalFlowFarneback(grays[mid - 1], g2, None, **fb_params)
        print(f"{name:<38} mean false motion = {np.linalg.norm(fl_b[250:290, 250:390], axis=2).mean():.3f} px")

# ------------------------------------------------------------
# Step 9: Display and save
# ------------------------------------------------------------
rgb = lambda x: cv2.cvtColor(x, cv2.COLOR_BGR2RGB)
fig, ax = plt.subplots(2, 3, figsize=(15, 8))
ax[0, 0].imshow(rgb(frames[0])); ax[0, 0].set_title("Frame 1")
ax[0, 1].imshow(rgb(frames[mid])); ax[0, 1].set_title(f"Frame {mid + 1}")
ax[0, 2].imshow(rgb(lk_vis)); ax[0, 2].set_title("Lucas-Kanade: trajectories of tracked points")
ax[1, 0].imshow(rgb(flow_color)); ax[1, 0].set_title("Farneback: colour = direction, brightness = speed")
ax[1, 1].imshow(rgb(arrows)); ax[1, 1].set_title("Farneback: flow vectors (arrows)")
ax[1, 2].imshow(flow_mag, cmap="hot"); ax[1, 2].set_title("Farneback: motion magnitude")
for a in ax.ravel(): a.axis("off")
plt.tight_layout(); plt.savefig("output/exp7_optical_flow.png", dpi=150); plt.show()

# Optional: save a colour-coded dense flow video
out = cv2.VideoWriter("output/exp7_dense_flow.mp4", cv2.VideoWriter_fourcc(*"mp4v"), 15,
                      (grays[0].shape[1], grays[0].shape[0]))
for i in range(1, len(grays)):
    fl = cv2.calcOpticalFlowFarneback(grays[i - 1], grays[i], None, **fb_params)
    out.write(flow_to_color(fl)[0])
out.release()
print("\nSaved: output/exp7_optical_flow.png and output/exp7_dense_flow.mp4")
```

Questions and Answers

1. What is Optical Flow? How is it used in computer vision?

Optical flow is the pattern of apparent motion of brightness patterns in an image sequence, caused by the relative motion between the camera and the scene. It is represented as a vector field (u, v) giving the displacement of each pixel (or point) between consecutive frames. It is used for:

- Object tracking and motion detection
- Video stabilization and frame interpolation
- Estimating camera ego-motion and obstacle avoidance
- Action and gesture recognition, video compression, and 3D structure from motion

2. Explain the working principle of the Lucas-Kanade Optical Flow algorithm.

Lucas-Kanade estimates the motion of a small set of points. It assumes brightness constancy, small motion between frames, and that all pixels in a small window around the point move together (spatial coherence).

1. From brightness constancy and a first-order Taylor expansion, each pixel gives $I_x\, u + I_y\, v = -I_t$.
2. A single equation cannot determine both u and v (aperture problem), so the equations from all pixels in a window (e.g., 21 x 21) are stacked.
3. The overdetermined system is solved by least squares to obtain (u, v) for the point.
4. Points are chosen where the window has strong texture or corners (Shi-Tomasi, `goodFeaturesToTrack`), and an image pyramid (`maxLevel`) handles larger motions.
5. In OpenCV: `cv2.calcOpticalFlowPyrLK(prev, next, points, None, winSize, maxLevel, criteria)`.

3. What is Dense Optical Flow? How does it differ from Sparse Optical Flow?

Aspect	Sparse Optical Flow	Dense Optical Flow
Pixels processed	Only selected feature points (corners)	Every pixel in the image
Output	Motion vectors for a few points	A flow vector (u, v) for each pixel, shown as a flow field or color map
Speed	Fast	Slower, more memory
Example algorithms	Lucas-Kanade	Farneback, Horn-Schunck, TV-L1, deep networks (RAFT)
Typical use	Point tracking, feature tracking, visual odometry	Motion segmentation, video analysis, action recognition, interpolation

4. Compare the Lucas-Kanade and Farneback optical flow algorithms.

Aspect	Lucas-Kanade	Farneback
Type	Sparse, feature-based	Dense, whole image
Principle	Least-squares solution of the flow constraint in a local window	Polynomial expansion of pixel neighborhoods and displacement estimation, with pyramids
Accuracy	Accurate at well-textured corners; unreliable elsewhere	Gives motion everywhere, but smoother and less accurate at boundaries and in flat regions
Complexity	Low; scales with number of points	Higher; scales with the number of pixels
Robustness	Fails if points are lost, occluded, or have large motion (needs pyramid)	Handles large motion with pyramids, but noisy in textureless areas
Applications	Object and feature tracking, real-time systems	Motion segmentation, scene understanding, video analysis

5. What assumptions are made while computing optical flow?

- **Brightness constancy:** a pixel's intensity does not change as it moves between frames
- **Small motion:** displacement between frames is small so the first-order Taylor expansion is valid (pyramids relax this)
- **Spatial coherence:** neighboring pixels belong to the same surface and move similarly (used by Lucas-Kanade's window and by smoothness terms in other methods)
- **Temporal persistence:** motion changes gradually over time and frames are captured at a sufficiently high frame rate

6. What factors can affect the accuracy of optical flow estimation?

- Large object speed or low frame rate (violates small motion)
- Illumination changes, shadows, and reflections (violate brightness constancy)
- Lack of texture (aperture problem) and repeated patterns
- Occlusions and disocclusions, motion blur, and noise
- Camera motion and rotation, and non-rigid or transparent objects
- Algorithm parameters: window size, pyramid levels, smoothing

7. Mention five real-world applications of optical flow in computer vision.

1. Object tracking and motion detection in video surveillance
2. Autonomous driving and ADAS: ego-motion estimation and obstacle detection
3. Robot and drone navigation (visual odometry, obstacle avoidance)
4. Action, gesture, and sports analytics (motion analysis of players)
5. Video stabilization, video compression, and frame-rate interpolation; also augmented reality and medical imaging (cardiac motion)

8. Why are grayscale images generally used for optical flow computation?

Optical flow is based on how image intensity changes over space and time, so a single intensity channel is sufficient to compute the gradients I_x, I_y, I_t. Using grayscale reduces computation and memory to one third compared with three color channels, avoids having to combine inconsistent per-channel estimates, and the brightness constancy assumption is defined for a single intensity value. Color can add information, but at a higher cost and with sensitivity to color changes from lighting.

9. What are the limitations of optical flow algorithms in real-world environments?

- Sensitive to lighting changes, shadows, and reflections
- Poor results in textureless or repetitive regions (aperture problem)
- Large or fast motion, motion blur, and occlusions cause errors
- Noise sensitivity, and accuracy drops at object boundaries (over-smoothing in dense methods)
- Camera motion must be separated from object motion
- Dense methods are computationally expensive for real-time use on limited hardware

10. How does optical flow contribute to applications such as autonomous driving, video surveillance, and action recognition?

- **Autonomous driving:** estimates ego-motion and the motion of other vehicles and pedestrians, detects obstacles and estimates time to collision, and supports visual odometry and lane tracking.
- **Video surveillance:** detects moving objects against the background, tracks people and vehicles, and helps flag unusual movement or crowd behavior.
- **Action recognition:** flow fields give motion information separate from appearance; two-stream networks use optical flow as input alongside RGB frames to recognize actions and gestures.

Conclusion

Lucas-Kanade sparse optical flow and Farneback dense optical flow were implemented using Python and OpenCV. Lucas-Kanade is fast and well suited to tracking specific feature points, whereas Farneback gives a motion estimate for every pixel and is suited to motion segmentation and scene analysis, at a higher computational cost. Both rely on brightness constancy and small motion, so image pyramids, good texture, and stable lighting are important for accurate results.